Cadence

Code Review

Generated May 25, 2026 · Claude Code (claude-sonnet-4-6)

Overview

Cadence is a SwiftUI health-tracking app with daily symptom logging, weekly reflection reviews, pattern-detection insights, HealthKit integration, and StoreKit 2 monetisation. The codebase follows a clean MVVM structure using SwiftData for persistence across five feature modules.

Overall the code is readable and well-organised. The issues below fall into three categories: real bugs that affect user-visible behaviour, architectural inconsistencies that will cause maintenance pain, and smaller code-quality items.

Category	Count	Items
Bug	5	#1 #2 #3 #4 #5
Architecture	3	#6 #7 #8
Code Quality	7	#9 – #15

Bugs

[BUG] #1 Pro status has two independent sources of truth

Files: `App/AppState.swift:6`, `Services/StoreService.swift:48`

AppState.isPro (UserDefaults-backed) and **StoreService.isPro** (computed from `purchasedProductIDs`) never synchronise. If **restorePurchases()** succeeds, **AppState.isPro** remains `false`. Conversely, calling **AppState.unlockPro()** never updates `StoreService.purchasedProductIDs`. Views that read from different sources will disagree on the user's pro state.

Fix: remove `isPro` from **AppState** and drive all gating from `StoreService.isPro`. Inject **StoreService** as an environment object where needed.

[BUG] #2 HealthKit sleep data is fetched but never stored

File: `Services/HealthKitService.swift:40`

fetchYesterdayData() returns a `HealthKitSnapshot` containing `sleepHours`, but **populate(log:)** only writes `steps`, `restingHR`, and `hrv` to the log. Sleep data is silently discarded. **DailyLog** has no `hkSleepHours` field, so the missing assignment isn't caught at compile time.

Fix: add `var hkSleepHours: Double?` to **DailyLog** and assign it in **populate(log:)**, or remove sleep from the HealthKit fetch entirely.

[BUG] #3 PatternEngine.analyze is always called with an empty array

File: `Features/DailyLog/DailyLogViewModel.swift:31`

```
PatternEngine.shared.analyze(logs: [], context: context)
```

The `logs: []` argument is hardcoded empty, so the engine never receives real data on save. The **analyze()** function body is also a stub (comment only, no implementation). Pattern insights therefore only appear when views explicitly call **allInsights(from:)** with live query results.

Fix: pass the full log array from the model context, or remove the **analyze()** call until the stub is implemented.

[BUG] #4 stressFatiguePattern checks the wrong day

File: `Services/PatternEngine.swift:70`

When a stress streak ends at index `i`, the code checks `sorted[i].symptoms` for fatigue — that is the *last high-stress day*, not the day after. The stated pattern ("fatigue tends to follow") would require checking `sorted[i + 1]` with a bounds guard. Additionally the confidence is hardcoded at 0.72 while the other three detectors compute real values from data.

Fix: guard `i + 1 < sorted.count` and check `sorted[i + 1].symptoms`. Derive confidence from `fatigueFollowed / occurrences`.

[BUG] #5 PDF doctor report has no page overflow protection

File: `Features/Export/PDFBuilder.swift:26`

renderPersonalSummary guards against overflow with `if y > 750 { ctx.beginPage(); y = 40 }`, but **renderDoctorReport** has no equivalent check. A user with many distinct symptoms will have content drawn past the 842pt page boundary and silently clipped. The personal summary guard also only handles a single overflow and will clip responses longer than 60pt.

Fix: add the same overflow guard to the doctor report, and measure actual text height before drawing rather than assuming fixed heights.

Architecture

[ARCHITECTURE] #6 AppState.currentStreak is dead state

File: `App/AppState.swift:8`

DashboardViewModel computes its own `streak: Int` and never writes it back to `AppState.currentStreak`. **NotificationService.scheduleStreakAtRisk()** schedules its notification unconditionally every day without checking this value. The published property exists but is never read or written by any view or service.

Fix: either remove `currentStreak` from **AppState**, or write the computed streak back to it from **DashboardViewModel.refresh()** and use it to conditionally schedule the streak-risk notification.

[ARCHITECTURE] #7 WeeklyReview has duplicate field storage

File: `Models/WeeklyReview.swift:8`

`wins`, `misses`, `energyAudit`, and `nextWeekIntentions` are stored as separate String fields alongside `promptResponses: [PromptResponse]`, which covers the same content via the weekly prompts. The review views render only `promptResponses`, making the four separate fields dead SwiftData storage that adds schema surface area without benefit.

Fix: remove the four separate fields and read everything from `promptResponses`, or make them the canonical store and remove the overlap from `promptResponses`.

[ARCHITECTURE] #8 Color.metricColor ignores self

File: `Shared/Extensions/View+Extensions.swift:28`

```
extension Color {
    func metricColor(for value: Int, inverted: Bool = false) -> Color
```

This is an instance method on **Color** that completely ignores `self` and always returns a new colour. Callers must supply a dummy receiver (`Color.clear.metricColor(for: 7)`), which is misleading and confusing.

Fix: change to `static func metricColor(for value: Int, inverted: Bool = false) -> Color`.

Code Quality

[CODE QUALITY] #9 try! crash risk at launch

File: `App/CadenceApp.swift:12`

`try! ModelContainer(...)` will crash on any SwiftData schema migration failure — a common occurrence when adding or renaming model fields across app versions. There is no recovery path.

Fix: Wrap in `do/catch` and fall back to an in-memory store or show an error screen.

[CODE QUALITY] #10 Force-unwraps in HealthKitService.readTypes

File: `Services/HealthKitService.swift:11`

All six `HKObjectType` lookups in `readTypes` use `!` at property initialisation time, crashing if any identifier is unavailable. The private helpers (**fetchSum**, **fetchLatest**) correctly use `guard let` — inconsistent style.

Fix: Use `compactMap` to build the set, dropping any unavailable types gracefully.

[CODE QUALITY] #11 completionScore penalises default values

File: `Models/DailyLog.swift:46`

Conditions like `mood != 5 || energy != 5` and `sleepHours != 7.0` mean a user with a genuinely average day (mood 5, 7 hrs sleep) scores lower than one who just dragged a slider. The score measures slider interaction, not log completeness.

Fix: Track whether each field was explicitly touched using a `didEditMetrics` flag set in the log flow, or always consider filled metrics as complete.

[CODE QUALITY] #12 Notification authorization result is silently ignored

File: `Services/NotificationService.swift:8`

The `requestAuthorization` completion handler discards both the `granted` bool and the `error`. Settings cannot reflect whether notifications are enabled, and denied users cannot be prompted to re-enable.

Fix: Store the result (e.g., in **AppState**) so the Settings view can show current permission status.

[CODE QUALITY] #13 Streak-risk notification fires every day unconditionally

File: `Services/NotificationService.swift:43`

`scheduleStreakAtRisk()` sets a repeating daily trigger regardless of whether the user has already logged today. Users who log in the morning still receive a “don’t break your streak” alert at 9 pm.

Fix: Cancel the notification after each successful save, and only schedule it on days where `todayLog?.isComplete` is false.

[CODE QUALITY] #14 WeeklyReview date range inconsistent with weekEndDate

File: `Features/WeeklyReview/WeeklyReviewViewModel.swift:32`

populateSummary filters `log.date < weekStartDate + 7 days`, while `WeeklyReview.weekEndDate` returns `weekStartDate + 6 days`. The 7th day is included in aggregates but excluded from the display label.

Fix: Align both to the same boundary — either `+6` days inclusive or `< start + 7` throughout.

[CODE QUALITY] #15 weekLabel allocates a DateFormatter on every access

File: `Models/WeeklyReview.swift:41`

`weekLabel` is a computed property called repeatedly in list views. It allocates and configures a `DateFormatter` each time, which is expensive.

Fix: Use `date.formatted(.dateTime.month(.abbreviated).day())` (no allocation) or a file-scope static formatter.

The three highest-priority fixes are: **#1** (pro status split source of truth — users who restore purchases may not get pro access), **#2** (HealthKit sleep data silently discarded), and **#5** (PDF overflow clips doctor reports for power users).